

HLD Case Study 3: Instagram (Photo Sharing and Feed)

Key Focus Areas: Media upload and processing pipeline, feed generation (fan-out on write vs read), CDN delivery, and storage cost.

1. Requirements and Scope

Functional: upload photos/videos, follow users, home feed of followed users' posts, likes and comments, explore/search. **Non-functional:** feed loads in under 500 ms p99; uploads never lose data; the feed may be seconds stale (eventual consistency); high availability; global low-latency media. **Out of scope:** ads, stories expiry mechanics, ML ranking internals.

2. Estimates

Assume 1B MAU, 500M DAU, 100M new posts/day (average 2 MB original; derived renditions add about 1.5x), each DAU opens the feed 10 times/day.

Takeaways: reads outnumber writes by about 50:1 (cache and CDN everything); media storage is the dominant cost (about 182 PB/year), so use object storage with lifecycle tiering and image compression; egress from feed images alone is about 1.9 Tbps (the code above), and more with video, which only a CDN can serve.

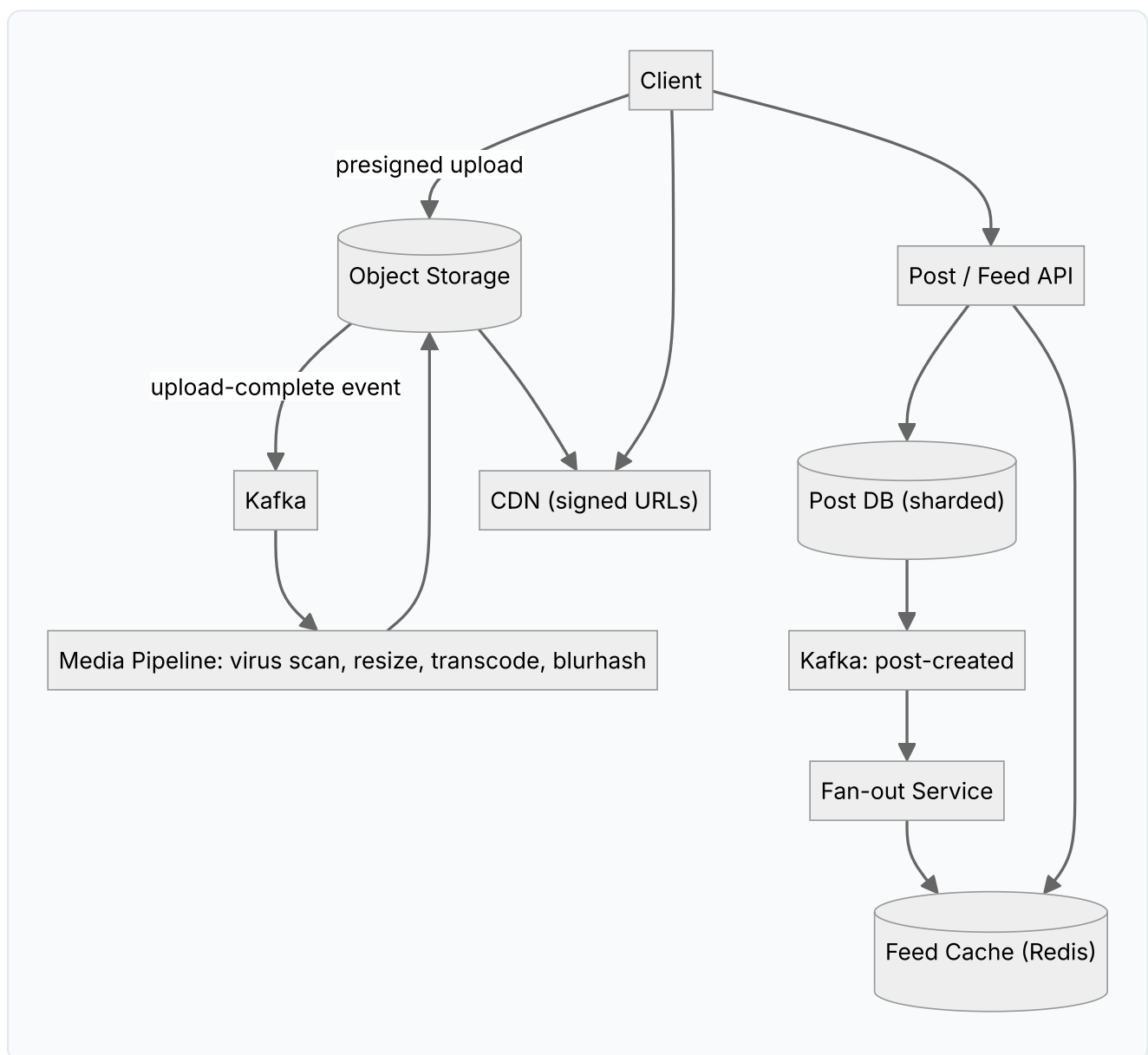
3. API Design

- `POST /media/uploads` returns presigned URL(s) (multipart for large files) and `media_id`.
- `POST /posts {media_id, caption, idempotency_key}` creates the post after the upload completes.
- `GET /feed?cursor=&limit=20` returns post ids with metadata and signed CDN URLs.
- `POST /posts/{id}/like`, `POST /users/{id}/follow`.

4. Data Model

- **posts:** `post_id` (time-sortable id such as Snowflake), `author_id`, `media_keys`, `caption`, `created_at`; sharded by `author_id` or by `post_id` range.
- **follows:** (`follower_id`, `followee_id`) stored both directions for "who do I follow" and "who follows me" (graph store or sharded relational).
- **feed cache:** `feed:{user_id}` as a Redis sorted set of the latest 500 `post_id`s scored by time.
- **likes:** counters in Redis, periodically flushed; per-user like edge in a KV store.

5. Architecture



6. Deep Dive: Feed Generation

Fan-out on write (push). When a user posts, a worker prepends `post_id` to the feed cache of every follower. Reading the feed is a single Redis range read: very fast. Cost: a user with 50M followers causes 50M writes per post. **Fan-out on read (pull).** At read time, fetch recent posts from each followee and merge. Cheap writes, slow reads for users who follow many accounts. **Hybrid (what production systems do).** Push for normal users (under about 10K followers); **do not push for celebrities.** At read time, merge the user's precomputed feed with a live pull of recent posts from the celebrities they follow. Then apply ranking (affinity, recency, engagement) on about 500 candidates.

Media pipeline. Upload goes straight to object storage via presigned URLs (the API tier never carries bytes). An event triggers asynchronous workers that scan, strip EXIF, generate renditions (for example 150, 320, 640, 1080 px) plus WebP/AVIF, and extract a blur placeholder. The post becomes visible only when processing marks it `READY`.

7. Scaling and Bottlenecks

- **Celebrity hot keys:** hybrid fan-out; cache hot posts and counters with request coalescing.
- **Fan-out lag:** use a queue with worker autoscaling; accept seconds of staleness; prioritise active users first.
- **Storage cost:** tier old media to infrequent-access classes, store only needed renditions, deduplicate identical uploads by content hash.
- **Like counters:** write to Redis `INCR`, batch to the database; counts are approximate for display.
- **Cold start feed:** if the cache is empty (new device or evicted), rebuild by pulling and merging, then repopulate.

8. Failure Modes and Mitigations

Failure	Mitigation
Upload interrupted	resumable multipart upload; <code>idempotency_key</code> on post creation
Media worker crash	queue redelivery; processing is idempotent keyed by <code>media_id</code>
Feed cache loss	rebuild from DB via pull; feeds are derived data
CDN regional failure	multi-CDN or origin fallback
Poison media (corrupt file)	dead-letter queue, mark post failed, notify user

9. Trade-offs

- Push gives $O(1)$ reads but write amplification; pull gives $O(1)$ writes but expensive reads; hybrid bounds both.
- Strong consistency for follows/posts is unnecessary; eventual consistency keeps latency low.
- Precompute renditions (storage cost) vs on-the-fly resizing at the edge (CPU cost and latency).
- Chronological vs ranked feed: ranking raises engagement but needs features, models and explainability.

10. Interview Timeline and Follow-ups

Lead with the read:write ratio, then the fan-out decision, then the celebrity edge case. **Follow-ups:** How do you delete a post everywhere? (tombstone, async removal from caches and CDN purge). How do you implement "Explore"? (offline candidate generation plus an ANN index, course 10). How do you shard posts? (by `author_id` keeps profile pages local; by `post_id` spreads writes evenly). How would you support stories that expire in 24 h? (TTL in storage and cache, lifecycle policy).

5. Runnable Model: Fan-out on Write, Fan-out on Read, and the Hybrid

The feed design stands or falls on one trade-off, and a small simulation shows why celebrities force a hybrid.



The three modes return the same feed but cost differently: **push** makes reads trivial and a celebrity post costs thousands of writes; **pull** makes writes free and every read merges many authors; **hybrid** pushes for ordinary users and pulls celebrity posts at read time, which bounds both costs. This is the design in the architecture section above, shown as numbers.

6. Failure Modes and Mitigations

Failure	Effect	Mitigation
Fan-out queue backlog	New posts appear late in followers' feeds	Prioritise active followers first, scale workers, degrade to pull for lagging users
Media upload succeeds, metadata write fails	Orphan blob	Write metadata after upload and garbage-collect unreferenced blobs, or use a pending state
Cache loss for a popular feed	Thundering herd on the database	Request coalescing, replicas, staggered TTLs
CDN origin overload from a viral post	Slow images	Tiered CDN, origin shield, pre-warm for known big accounts
Unfollow or delete after fan-out	Stale items in inboxes	Filter at read time against current follow and deletion state

7. Trade-offs and Alternatives

- **Ranked versus chronological feed:** chronological is cheap and predictable; ranking needs candidate generation, features and a model, and moves cost to read time.
- **Inbox size:** cap each precomputed inbox (for example the newest 1,000 items) so storage per user is bounded and old content is fetched by pull.
- **Counts (likes, followers):** exact counters are expensive at scale; approximate or sharded counters with periodic aggregation are normal.
- **Storing media:** originals in an object store, several resized variants generated asynchronously, everything behind a CDN.

8. Interview Timeline (45 minutes) and Follow-ups

Minutes	Do
0 to 5	Scope: feed, upload, follow, likes; scale; ranked or chronological
5 to 10	Estimates: uploads per day, feed reads per second, storage and egress
10 to 20	APIs, data model, upload and media pipeline
20 to 35	Feed generation: push, pull, hybrid and the celebrity problem
35 to 42	Caching, CDN, counters, sharding
42 to 45	Failure modes and what you would improve next

Follow-ups to prepare: What is the cost of one celebrity post under push? How do you pick the celebrity threshold? How do you add ranking without making reads slow? How do you handle deletes and privacy changes already fanned out?